ONLINE MASTERS IN **DATA SCIENCE**

DSC 257R - UNSUPERVISED LEARNING

WORD EMBEDDINGS

SANJOY DASGUPTA, PROFESSOR

UC San Diego
COMPUTER SCIENCE & ENGINEERING
HALICIOĞLU DATA SCIENCE INSTITUTE

Word Embeddings

Represent each word by a vector in $\mathbb{R}^d$, so that words with similar meanings have vectors that are close together.

Word2Vec: Analogy Problems

Analogy problem: `king is to queen as man is to?`

Word2Vec: Analogy Problems

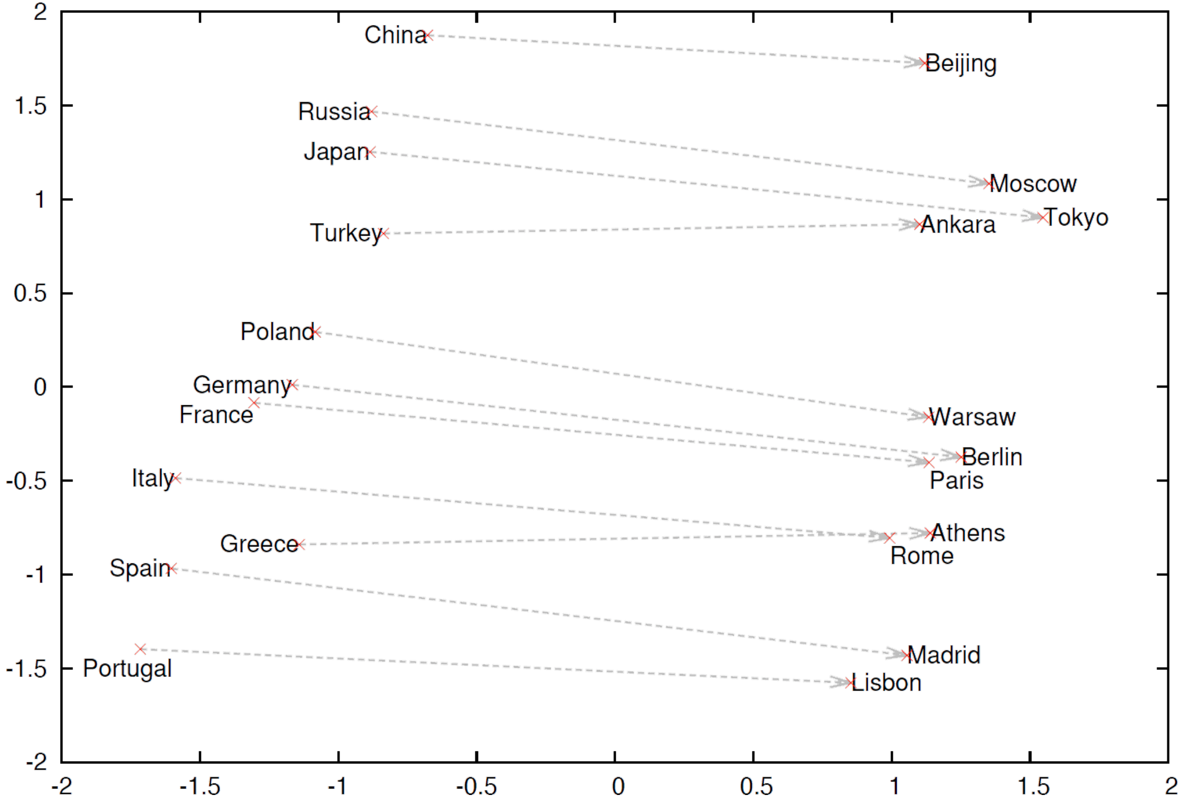
Analogy problem: `king is to queen as man is to?`

Treat this as a linear equation:

- vec(king) – vec(queen) = vec(man) – vec(?)
- vec(?) = vec(woman) – vec(king) + vec(queen)
- Nearest neighbor of this vector is vec(woman)

Countries and Capitals

Country and Capital Vectors Projected by PCA



Meaning from Context

You shall know a word by the company it keeps. (J.R. Firth, 1957)

Meaning from Context

You shall know a word by the company it keeps. (J.R. Firth, 1957)

- Much of the meaning of a word w is captured by the *contexts in which it appears:*

$$w_1 \quad w_2 \quad w_3 \quad w \quad w_4 \quad w_5 \quad w_6$$

- Find an embedding of words based on this insight.

Prediction problem: **predict** w **from its context** (or vice versa).

Word2Vec: Skip-Gram Formulation

- Vocabulary W
- For each word, **two embeddings** $\alpha_w, \beta_w \in \mathbb{R}^d$.

Word2Vec: Skip-Gram Formulation

- Vocabulary W
- For each word, **two embeddings** $\alpha_w, \beta_w \in \mathbb{R}^d$.
- Probabilistic model: for any words w_o, w_c,

$$\Pr(w_c \text{ appears in context of } w_o) = \frac{e^{\alpha_{w_c} \cdot \beta_{w_o}}}{\sum_w e^{\alpha_w \cdot \beta_{w_o}}}$$

Word2Vec: Skip-Gram Formulation

- Vocabulary W
- For each word, **two embeddings** $\alpha_w, \beta_w \in \mathbb{R}^d$.
- Probabilistic model: for any words w_o, w_c,

$$\Pr(w_c \text{ appears in context of } w_o) = \frac{e^{\alpha_{w_c} \cdot \beta_{w_o}}}{\sum_w e^{\alpha_w \cdot \beta_{w_o}}}$$

- Learning problem: find vectors $\{\alpha_w, \beta_w : w \in W\}$ that maximize

$$\sum_{t=1}^{T} \sum_{-c \le j \le c, j \ne 0} \log \Pr\left(w_{t+j} \middle| w_t\right).$$

Word2Vec: Skip-Gram Formulation

- Vocabulary W
- For each word, **two embeddings** $\alpha_w, \beta_w \in \mathbb{R}^d$.
- Probabilistic model: for any words w_o, w_c,

$$\Pr(w_c \text{ appears in context of } w_o) = \frac{e^{\alpha_{w_c} \cdot \beta_{w_o}}}{\sum_w e^{\alpha_w \cdot \beta_{w_o}}}$$

- Learning problem: find vectors $\{\alpha_w, \beta_w : w \in W\}$ that maximize

$$\sum_{t=1}^{T} \sum_{-c \leq j \leq c, j \neq 0} \log \Pr\left(w_{t+j} \middle| w_t\right).$$

- Issue: computing $\nabla \log p(w_c | w_o)$ takes time $O(|W|)$.

Word2Vec: Negative Sampling

Pick a noise distribution Q over W.

For each observed (word, context) pair (w_o, w_c):

- Draw k noise samples $w_1, \dots, w_k \sim Q$
- Create one positive example (w_o, w_c)
- Create k negative examples $(w_o, w_1), \dots, (w_o, w_k)$

This leads to the following term in the optimization:

$$\log \sigma(\alpha_{w_c} \cdot \beta_{w_o}) + \sum_{i=1}^{k} \log \sigma(-\alpha_{w_i} \cdot \beta_{w_o}).$$